

Cryptography Fundamentals Lab Study Guide

This study guide prepares you for the Cryptography Lab. It covers encoding, hashing, encryption, and decryption. Read each section before starting the corresponding module. The examples use dummy strings to teach concepts. This ensures you learn the methods without spoiling the flags. No em-dashes are used in this document.

Table Of Contents

Module 1: Cryptography Basics

- Conceptual Background
- Key Terms and Data Formats
- Command Reference
- Worked Examples
 - Example 1: Decode a simple Base64 string
 - Example 2: Decode a hexadecimal string
- Before You Start This Module
- Common Mistakes
- Further Reading

Module 2: Hash Basics

- Conceptual Background
- Key Hashing Algorithms
- Command Reference
- Worked Examples
 - Example 1: Identify hash length
 - Example 2: Hash verification
- Before You Start This Module
- Common Mistakes
- Further Reading

Module 3: Hash Calculator

- Conceptual Background
- Hashing Suite Commands
- Worked Examples
 - Example: Verify a download
- Before You Start This Module

Common Mistakes

Further Reading

Module 4: Encryption Basics

Conceptual Background

Encryption Algorithms

Caesar Cipher

ROT13

XOR Encryption

Worked Examples

Example 1: Caesar Encryption

Example 2: XOR Encryption

Before You Start This Module

Common Mistakes

Further Reading

Module 5: Decryption Basics

Conceptual Background

Worked Examples

Example 1: Decrypt a Caesar Cipher

Example 2: Decrypt an XOR cipher

Before You Start This Module

Common Mistakes

Further Reading

Module 1: Cryptography Basics

This module introduces encoding systems. You will learn to decode Base64 and Hexadecimal values.

Conceptual Background

Encoding converts data into a specific format. This is done for system compatibility or safe transmission. Encoding does not hide information. It does not require a secret key. Anyone can decode the message easily. This differs from encryption, which uses a key to keep data secret. It also differs from hashing, which is a one-way process.

Key Terms and Data Formats

- **Base64**: Converts binary data into printable ASCII text. It uses characters A to Z, a to z, 0 to 9, +, /, and =. The equal sign (=) is used for padding.
- **Hexadecimal (Base16)**: Represents bytes using numbers 0 to 9 and letters a to f. Each byte is written as a pair of characters. For example, the character H has a hex value of 48.

Command Reference

You can decode these formats in the Linux terminal. You can also use the web interface decoder panel.

```
# Decode a Base64 string in the terminal
echo 'dGVzdF9zdHJpbmc=' | base64 -d
```

```
# Decode a hexadecimal string in the terminal
echo '746573745f737472696e67' | xxd -r -p
```

Worked Examples

Example 1: Decode a simple Base64 string

You have the string **Q29kZTIwMjY=**. Input this into the Base64 decoder. The output is **Code2026**.

Example 2: Decode a hexadecimal string

You have the hex string **536563757265**. Input this into the Hex decoder. The output is **Secure**.

Before You Start This Module

1. Is Base64 an encoding scheme or an encryption algorithm?
2. What character is typically used for padding at the end of a Base64 string?
3. How many hexadecimal characters are needed to represent a single byte?

Common Mistakes

- **Confusing encoding with encryption.** Encoding does not secure data. Do not use Base64 to hide passwords.
- **Including wrong characters in Hex decode.** Hexadecimal only uses characters 0 to 9 and a to f. Spaces or other letters will cause decoding failures.

Further Reading

- `man base64` (Base64 encoding and decoding command utility)
- `man xxd` (Hex dump utility)

Module 2: Hash Basics

This module covers the concepts of hashing and hash comparison.

Conceptual Background

A hash function takes an input of any size and produces a fixed-size string. This output is called a hash, a digest, or a checksum. Hashing is a one-way process. You cannot reverse a hash to find the original input. A specific input will always produce the exact same hash. Even a tiny change to the input changes the hash completely. This behavior is called the avalanche effect. Hashes verify data integrity and secure passwords.

Key Hashing Algorithms

- **MD5**: Produces a 128-bit digest. It is written as a 32-character hexadecimal string. It is no longer secure against collisions.
- **SHA-1**: Produces a 160-bit digest. It is written as a 40-character hexadecimal string.
- **SHA-256**: Produces a 256-bit digest. It is written as a 64-character hexadecimal string. It is highly secure and widely used.

Command Reference

Generate hashes in the terminal using these commands.

```
# Generate an MD5 hash
echo -n 'hello' | md5sum
```

```
# Generate a SHA-256 hash
echo -n 'hello' | sha256sum
```

Note: The `-n` flag prevents echo from adding a newline character. A newline will change the resulting hash value.

Worked Examples

Example 1: Identify hash length

You see a hash with 40 hexadecimal characters. This is a SHA-1 hash. A 64-character hexadecimal hash is a SHA-256 hash.

Example 2: Hash verification

You need to verify if two files are identical. Run `sha256sum file1.txt` and `sha256sum file2.txt`. Compare the outputs character by character. If they match exactly, the files are identical.

Before You Start This Module

1. Can you reverse a SHA-256 hash to retrieve the original password?
2. Why is the `-n` flag important when using `echo` to generate hashes?
3. Which hash algorithm produces a 32-character hex output?

Common Mistakes

- **Forgetting the newline character.** Running `echo "test" | md5sum` hashes "test\n" instead of "test". Always use `echo -n` for exact string hashing.
- **Case sensitivity errors.** Hashes are case-insensitive. However, matching scripts may expect lowercase letters.

Further Reading

- `man md5sum` (MD5 digest tool manual)
- `man sha256sum` (SHA-256 digest tool manual)

Module 3: Hash Calculator

This module focuses on calculating multiple hash digests for validation.

Conceptual Background

Systems administrators calculate digests to verify software downloads. If a downloaded file hash matches the vendor hash, the file is authentic. It has not been modified by an attacker. It was not corrupted during the download.

Hashing Suite Commands

You can calculate these values using the interactive calculator. You can also calculate them via the terminal.

```
# Calculate SHA-224 digest
echo -n 'data' | sha224sum
```

```
# Calculate SHA-384 digest
echo -n 'data' | sha384sum
```

```
# Calculate SHA-512 digest
echo -n 'data' | sha512sum
```

Worked Examples

Example: Verify a download

A vendor lists the SHA-256 hash for their software: `1a2b3c...`. You download the file to your workspace. You run `sha256sum software.tar.gz`. The output matches the vendor hash. The file is safe to install.

Before You Start This Module

1. What is the length of a SHA-512 digest in bits?
2. What are the key differences between the SHA-2 and SHA-3 families?

Common Mistakes

- **Hashing empty spaces.** Ensure no trailing spaces are included in your input field. Spaces change the digest output.

Further Reading

- `man sha512sum` (SHA-512 tool documentation)
- Review NIST hashing standards (Secure Hash Standard guidelines)

Module 4: Encryption Basics

This module covers classical encryption: Caesar, ROT13, and XOR.

Conceptual Background

Encryption changes readable text (Plaintext) into unreadable text (Ciphertext). It uses an algorithm and a key. Symmetric encryption uses the same key for encryption and decryption.

Encryption Algorithms

Caesar Cipher

An alphabet shift cipher. Each letter is shifted down the alphabet by a fixed number. This shift value is the key. For example, with a shift key of 1, A becomes B, and B becomes C.

ROT13

A special case of the Caesar cipher. It uses a fixed shift key of 13. Since the alphabet has 26 letters, applying ROT13 twice returns the original text.

XOR Encryption

A bitwise operation cipher. It compares binary bits of the plaintext against a key. If the bits are different, the output is 1. If the bits are the same, the output is 0. It is widely used in simple computer security scripts.

Worked Examples

Example 1: Caesar Encryption

Encrypt the word CAT with a shift key of 2.

- C shifted by 2 becomes E.
- A shifted by 2 becomes C.
- T shifted by 2 becomes V. The ciphertext is ECV.

Example 2: XOR Encryption

Encrypt A (hex 41) with XOR key 0x10. Hex 41 in binary is 01000001. Hex 10 in binary is 00010000. XOR operation output is 01010001 (hex 51). Hex 51 is the character Q.

Before You Start This Module

1. If you encrypt a message with ROT13, how many times must you run it to get the original text?
2. What is the ciphertext for **DOG** using a Caesar shift key of 3?
3. How is the XOR operation represented in standard logical math?

Common Mistakes

- **Over-shifting past Z.** In Caesar ciphers, the alphabet wraps around. Shifting Z by 1 brings you back to A.
- **Key formatting in XOR.** Ensure you distinguish between decimal keys and hex keys (starting with 0x).

Further Reading

- Read cryptography history guides on classical substitution ciphers.
- Look up bitwise XOR logical operations in programming reference books.

Module 5: Decryption Basics

This module focuses on reversing ciphers to recover plaintext.

Conceptual Background

Decryption is the reverse of encryption. It takes ciphertext and uses the key to reconstruct the plaintext. Cryptanalysis is the study of ciphers to find weaknesses. It allows codebreakers to decrypt messages without knowing the key.

Worked Examples

Example 1: Decrypt a Caesar Cipher

You have the ciphertext **KH00R** and know the shift key is 3. To decrypt, shift each letter backward by 3.

- K shifted back by 3 becomes H.
- H shifted back by 3 becomes E.
- O shifted back by 3 becomes L.
- O shifted back by 3 becomes L.
- R shifted back by 3 becomes O. The plaintext is **HELLO**.

Example 2: Decrypt an XOR cipher

XOR decryption uses the same process as encryption. Take the hex ciphertext **51** and apply the same key **0x10**. The output returns the original hex **41** (character **A**).

Before You Start This Module

1. How does knowing the encryption key simplify the decryption process?
2. What happens if you decrypt a Caesar cipher using the wrong shift value?

Common Mistakes

- **Decrypting in the wrong direction.** For Caesar decryption, you shift backward instead of forward.
- **Hex conversion errors.** Ensure you convert hex ciphertext back to binary before performing XOR calculations.

Further Reading

- Study basic cryptanalysis methods (frequency analysis of letters).
- Reference online documentation for the `crypt` library.
- Learn about common symmetric block cipher models.